

[R][HW] Generate Subsequences

i/p 30 20 10

(30, 10) ✓
(10, 30) ✗

10 20 30 40 50

10 40 50 ✓

20 30 40

Generate Subsequences

Given a sequence of integers, design an algorithm to generate all of its **subsequences**.

Note : We define a **subsequence** of a given sequence as a sequence which can be obtained by deleting **zero** or **more** elements from the given sequence such that the **relative order** between the sequence elements **doesn't change**.

Example

Input

0	1	2
10	20	30

10 20 30

(10) ✓ ✓

(10, 20) ✓ ✓

(10, 20, 30) ✓ ✓ (20) ✓ ✓ (10, 30) ✓ ✗

(20, 30) ✓ ✓

() ✓ ✗ (30) ✓ ✓

30, 10 ✗

Generate Pairs

$${}^nC_r = \frac{n!}{r!(n-r)!}$$

Generate Pairs

Given an array of N integers, design an algorithm to generate all of its **pairs**.

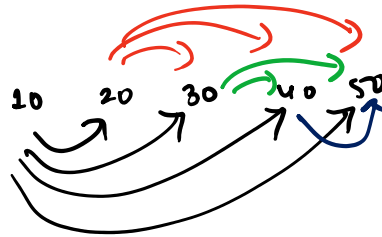
Example

Input

0	1	2	3	4
10	20	30	40	50

Output

$(10, 20), (10, 30), (10, 40), (10, 50),$
 $(20, 30), (20, 40), (20, 50),$
 $(30, 40), (30, 50),$
 $(40, 50)$



$${}^nC_2 = \frac{n!}{2!(n-2)!} = \frac{n(n-1)(n-2)!}{2(n-2)!} = \frac{n(n-1)}{2} \sim \frac{n^2}{2}$$

$$0 \leq i \leq n-2 \quad i+1 \leq j \leq n-1$$

i	j
0	1 to $n-1$
1	2 to $n-1$
2	3 to $n-1$
$\vdots$	$\vdots$
$n-2$	$n-1$ to $n-1$

#pairs

$n-1$
 $n-2$
 $n-3$
 $\vdots$
 1



$$\sum = \frac{n(n-1)}{2} \sim \frac{n^2}{2}$$

Target Sum Pairs

Target Sum Pairs

Given a **sorted** array containing **N distinct** integers, and a target integer **T**, design an algorithm to **count** the **number of pairs** of integers in the given array whose **sum is equal to T**.

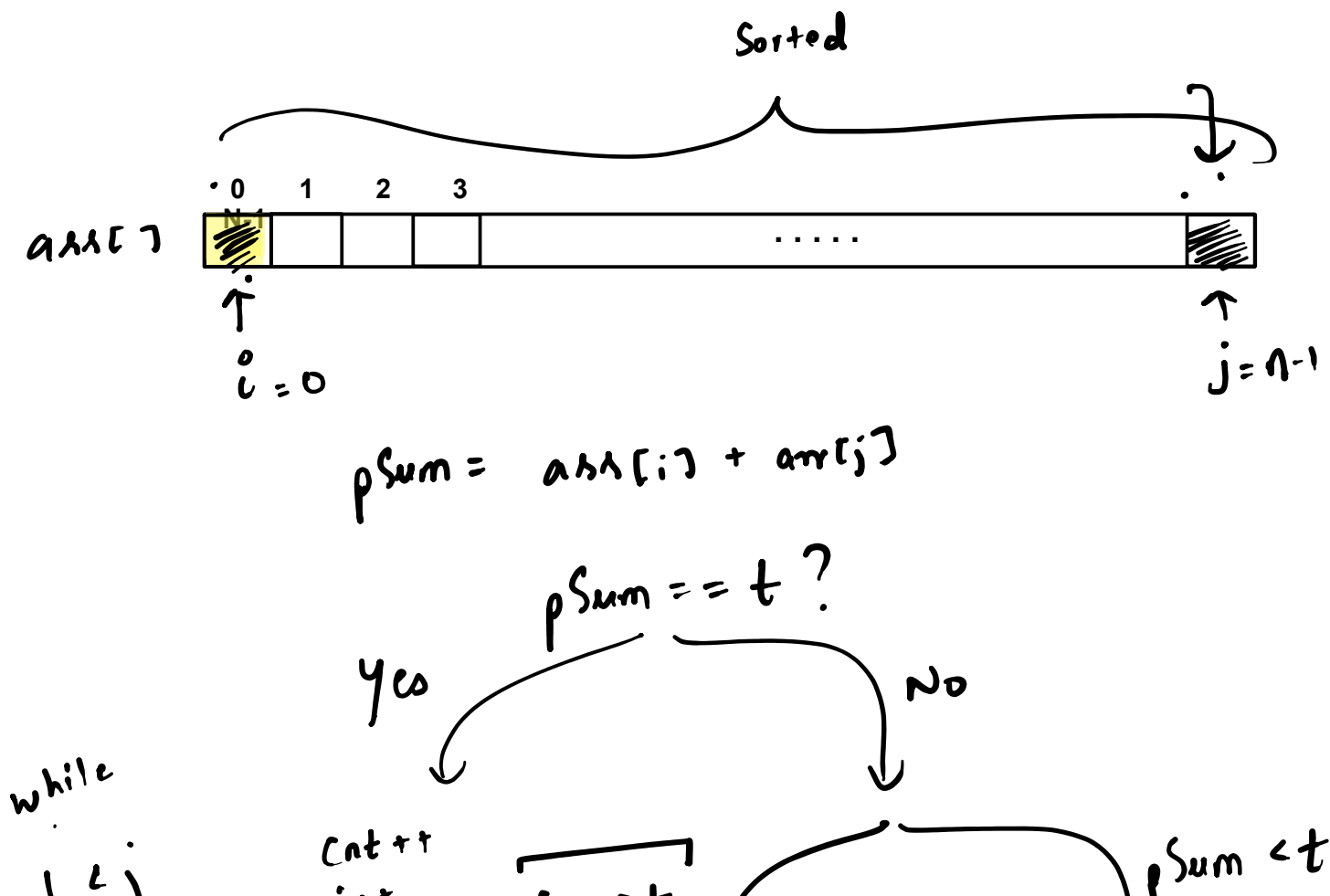
Example

Input

N = 6, T = 60

0	1	2	3	4	5
10	20	30	40	50	60

Output : 2



$i < j$

cnt++
i++
j--

$\text{Sum} > t$

j--

$\text{Sum} < t$

i++

i	i	i			
↓	↓	↓			
0	1	2	3	4	5
10	20	30	40	50	60
		↗	↗	↗	↗
		j	j	j	j

$t = 60$

$\text{cnt} = \cancel{0} / 2$

$$10 + 60 = 70 > t? \checkmark$$

$$10 + 50 = 60 == t? \checkmark$$

$$20 + 40 = 60 == t? \checkmark$$

$t = 1000$

10 20 30 40 50 60
 $\uparrow \quad \uparrow \quad \uparrow \quad \uparrow \quad \uparrow \quad \uparrow$
 $i=0 \quad i=1 \quad i=2 \quad i=3 \quad i=4 \quad i=5$
 $j=n-1$

$t = 50$

10 20 30 40
 $\uparrow \quad \uparrow \quad \uparrow \quad \uparrow$
 $i \quad i \quad j \quad j$

Sereja and Dima

Sereja and Dima play a game. The rules of the game are very simple. The players have n cards in a row. Each card contains a number, all numbers on the cards are distinct. The players take turns, Sereja moves first. During his turn a player can take one card: either the leftmost card in a row, or the rightmost one. The game ends when there is no more cards. The player who has the maximum sum of numbers on his cards by the end of the game, wins.

Sereja and Dima are being greedy. Each of them chooses the card with the larger number during his move.

Inna is a friend of Sereja and Dima. She knows which strategy the guys are using, so she wants to determine the final score, given the initial state of the game. Help her.

Input

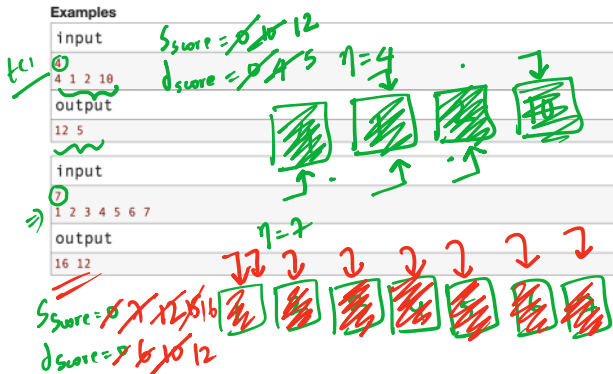
The first line contains integer n ($1 \leq n \leq 1000$) — the number of cards on the table. The second line contains space-separated numbers on the cards from left to right. The numbers on the cards are distinct integers from 1 to 1000.

Output

On a single line, print two integers. The first number is the number of Sereja's points at the end of the game, the second number is the number of Dima's points at the end of the game.

Examples

input	4 1 2 10	Copy
output	12 5	Copy
input	1 2 3 4 5 6 7	Copy
output	16 12	Copy



opt. problem

brute-force

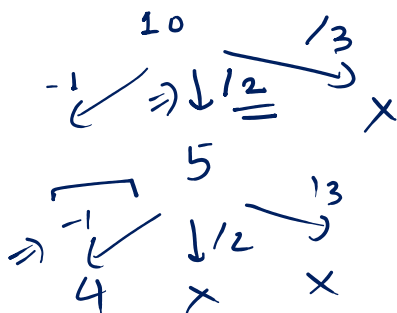
- exhaustive
- works
- slow

greedy

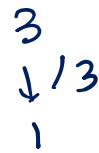
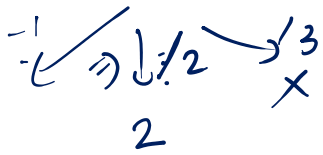
- selective
- faster
- may not work

10 → 1

$\left. \begin{array}{l} 1/2 \\ 1/3 \\ -1 \end{array} \right\}$



10
 ↓ -1
 9
 ↓ /3
 7

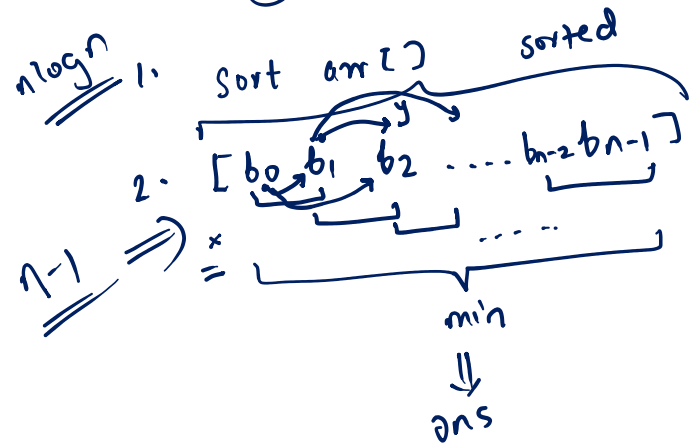


[a_0 a_1 a_2 a_{n-1}]

find min-diff b/w any two el.
in the given arr?

Brute-force

generate all pairs
for each pair find
the diff and
track the min diff } $O(n^2)$



Container with Most Water

Given an array of **N** integers representing heights of **N** vertical lines. Find two lines that together with the x-axis form a **container**, such that the container contains the **most water**. Return the **maximum** amount of water a container can store.

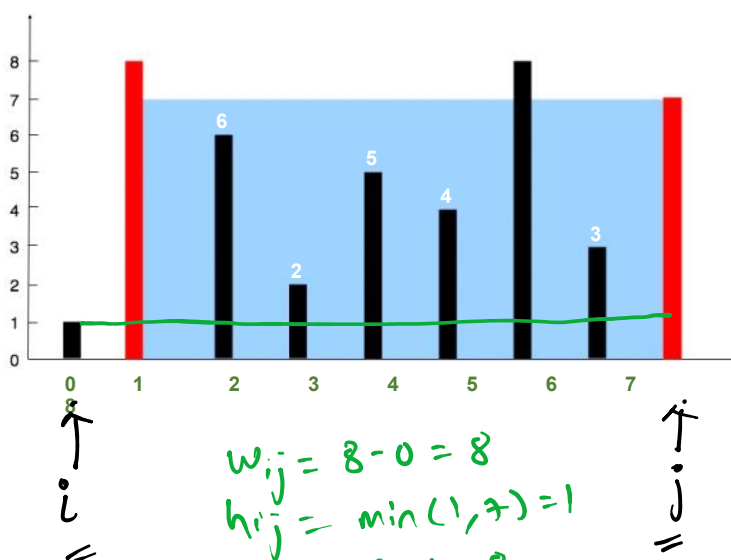
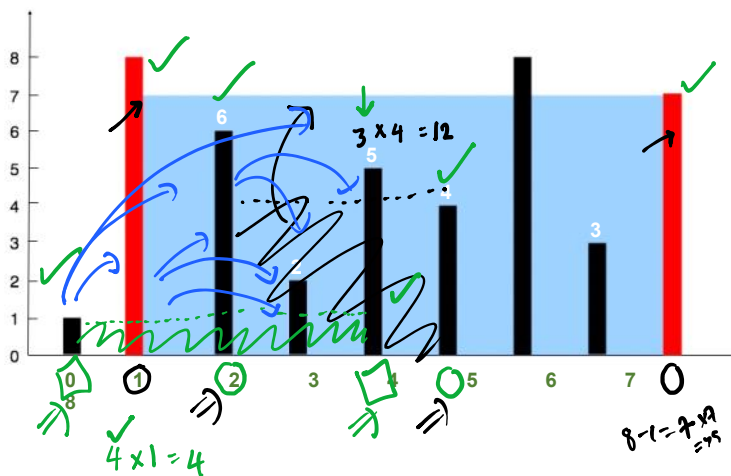
Example

Input

N = 9

0	1	2	3	4	5	6	7	8
1	8	6	2	5	4	8	3	7

Output : 49



maximize
 $\Downarrow$
 $area = \overbrace{width}^{width} * \underbrace{height}_{height}$

$a_{ij} = w_{ij} * h'_{ij}$
 $\Downarrow \quad \Downarrow$
 $j-i \quad \min(h_i, h_j)$
 $max_{sofar} = \max(max_{sofar}, a_{ij})$

if $h_i < h_j$ }

```
if  $h_i < h_j$ 
    i++
else
    j--
```



$$n \rightarrow n \log n$$

Merge Sorted Arrays

Given two **sorted** array consisting of **N** and **M** integers respectively, design an algorithm to merge the two sorted array such that the **merged** array is **sorted**.

Example

Input $n=4$ $m=3$

0	1	2	3	0	1	2
10	30	50	70	20	40	60

$a[]$ $b[]$

Output

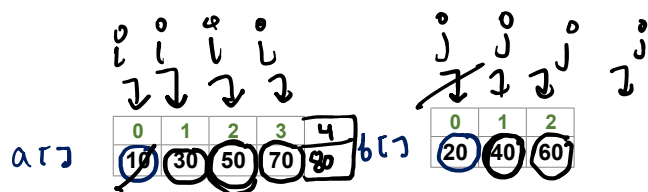
0	1	2	3	4	5	6
10	20	30	40	50	60	70

1. create a new array "c" of size $n+m$

2. transfer elements from $a[]$ and $b[]$ into $c[] \Rightarrow n+m$

3. sort $c[] \Rightarrow (n+m) \log(n+m)$

~~$(n+m) + (n+m) \log(n+m)$~~



$c[]$

0	1	2	3	4	5	6
10	20	30	40	50	60	

↑ ↑ ↑ ↑ ↑ ↑

R R R R R R

if ($a[i] < b[j]$)

$c[k] = a[i]$

$i++$

$k++$

} else {

$c[k] = b[j];$

$j++;$

$k++;$

}

while ($i < n-1$) {

$c[k] = a[i];$

$i++;$

$k++;$

}

$i < n-1$

and

$j < m-1$

```

/ ) {
    while (j <= m-1) {
        c[k] = b[j];
        j++;
        k++;
    }
}

```

⇒

Counting Smaller Elements

Given two **sorted** arrays consisting of **N** & **M** integers respectively, find out for each element in the first array the no. of elements smaller than that element in the second array.

Example

Input

0	1	2
4	6	8

0	1	2	3	4	5
1	2	3	5	7	9

Output

3	4	5
---	---	---

Merge Sorted Arrays in O(1) Space

Given a **sorted** array "A" of size **N+M** consisting of **N** integers & another sorted array "B" consisting of **M** integers, design an algorithm to merge the two sorted array such that the **merged array** is also **sorted without** using any **extra space**.

Example

Input

0	1	2	3	4	5	6
10	30	50	70			

0	1	2
20	40	60

Output

0	1	2	3	4	5	6
10	20	30	40	50	60	70